Developer Guide

This guide walks a new contributor through setting up a local environment, understanding the codebase, and making their first change. Read this before diving into any of the other docs in this folder.

What Is This Project?

DriveAbility is a review platform for assistive driving technology (hand controls, amputee rings, pedal extensions, etc.) built for the UA EcoCAR program. Reviewers publish methodology-backed reviews; users browse, filter, and save them. An admin dashboard manages content and users.

Prerequisites

Tool	Version	Notes
Node.js	v18+	Check with node -v
npm	v9+	Comes with Node
MongoDB Atlas account	—	Or a local mongod for offline dev
Git	any recent	—

No Docker is required for basic development (Docker configs exist for CI/deployment but are optional locally).

First-Time Setup

1. Clone and install

```
git clone <repo-url>
cd mc-website
npm run install-all
```

install-all runs npm install in the repo root, client/, and server/ in one shot. The root node_modules only contains concurrently (used to run both processes together).

2. Create your .env file

```
cp .env.example .env
```

Open .env and fill in at minimum:

```
MONGODB_URI=mongodb+srv://:@.mongodb.net/?retryWrites=true&w=majority
JWT_SECRET=
JWT_EXPIRE=30d
NODE_ENV=development
FRONTEND_URL=http://localhost:3000
```

The .env file lives at the **repo root** — both the server and the Vercel function read from it. There is no separate server/.env.

Email in dev: Leave email vars empty. The server auto-creates a free Ethereal test account and prints a preview URL to the terminal whenever an email would be sent. Nothing reaches a real inbox.

3. Start the dev servers

```
npm run dev
```

This runs both processes concurrently:

Process	URL	Command under the hood
Next.js frontend	http://localhost:3000	cd client && npm run dev
Express backend	http://localhost:5001	cd server && npm run dev (nodemon + ts-node)

The terminal output is prefixed — [0] is the server, [1] is the client (or vice versa depending on startup order).

4. Create an admin account

Register at http://localhost:3000/register, then promote yourself in MongoDB:

```
db.users.updateOne({ email: "you@example.com" }, { $set: { role: "admin" } })
```

The admin dashboard is at http://localhost:3000/admin.

Repository Layout

```
mc-website/
├── client/                 # Next.js 14 frontend (Pages Router)
│   └── src/
│       ├── components/     # Reusable UI components
│       │   ├── layout/     # Navbar, Footer
│       │   ├── reviews/    # Review cards, forms, filters
│       │   ├── products/   # Product cards, detail views
│       │   └── admin/      # Admin dashboard components
│       ├── contexts/       # React context providers (AuthContext)
│       ├── pages/          # File = route (Next.js Pages Router)
│       ├── styles/         # Global CSS
│       ├── types/          # Shared TypeScript interfaces
│       └── utils/          # API helper functions
│
├── server/                 # Express.js backend
│   └── src/
│       ├── config/         # db.ts — MongoDB connection
│       ├── controllers/    # Route handler logic (one file per resource)
│       ├── middleware/     # auth.ts (JWT), error handling
│       ├── models/         # Mongoose schemas
│       ├── routes/         # Express router definitions
│       ├── services/       # Business logic extracted from controllers
│       ├── migrations/     # One-off DB migration scripts
│       └── utils/          # devError, email helpers
│
└── api/
    └── index.ts            # Vercel serverless entry point (wraps Express app)
```

```
├── docs/                # Developer documentation (you are here)
├── vercel.json          # Rewrites /api/* → serverless function
└── .env.example         # Template for your local .env
└── package.json         # Root scripts: dev, install-all
```

How the Two Environments Differ

Understanding this saves a lot of debugging time:

	Development	Production (Vercel)
Frontend	next dev on port 3000	Built and served by Vercel
Backend	nodemon on port 5001	Single serverless function at api/index.ts
API calls	http://localhost:5001/api/... (env var or relative)	/api/... rewritten by vercel.json to the function
.env source	Repo root .env file	Vercel project dashboard env vars

The key insight: in production there is no port 5001. The client uses relative /api/ URLs and Vercel's rewrite rule sends them to the serverless function. In development, the Next.js dev server proxies or the client calls localhost:5001 directly — check client/src/utils/api.ts for how the base URL is determined.

Making a Backend Change

Adding a new API route

1. **Model** (if needed): server/src/models/MyThing.ts — define a Mongoose schema and export the model.
2. **Controller**: server/src/controllers/myThingController.ts — write async handler functions. Use the existing error pattern: typescript import { devError } from '../utils/devError'; export const getMyThing = async (req, res) => { try { const thing = await MyThing.findById(req.params.id); res.json({ success: true, data: thing }); } catch (err) { res.status(500).json({ success: false, message: 'Server error', ...devError(err) }); } };

3. **Route**: server/src/routes/myThingRoutes.ts — create an Express router, apply authenticate/authorize middleware as needed: typescript import { authenticate, authorize } from '../middleware/auth'; router.get('/:id', getMyThing); router.post('/', authenticate, authorize('admin'), createMyThing);
4. **Mount**: In server/src/app.ts, import the router and app.use('/api/mythings', myThingRoutes).
5. **Register model**: If you added a new model, import it in server/src/app.ts so Mongoose registers it on startup.

Auth middleware quick reference

```
import { authenticate, authorize } from '../middleware/auth';

// Require any logged-in user:
router.get('/profile', authenticate, getProfile);

// Require a specific role:
router.delete('/:id', authenticate, authorize('admin'), deleteItem);

// Multiple allowed roles:
router.post('/', authenticate, authorize('reviewer', 'admin'), createReview);
```

authenticate attaches the decoded JWT payload to req.user. authorize reads req.user.role.

Making a Frontend Change

Adding a new page

Create a file in client/src/pages/. The filename becomes the URL:

```
pages/my-feature.tsx          →  /my-feature
pages/products/[id].tsx       →  /products/:id  (dynamic)
pages/admin/my-section.tsx    →  /admin/my-section
```

Calling the API from a page

Use the utility helpers in client/src/utils/ rather than raw fetch. They handle the base URL and attach the auth header automatically. Check the existing helper to see what's available before writing a new fetch call.

Protecting a page (auth guard)

There is no automatic redirect middleware on the client. Add the check yourself at the top of the component:

```
import { useAuth } from '../contexts/AuthContext';
import { useRouter } from 'next/router';
import { useEffect } from 'react';

export default function ProtectedPage() {
  const { user, loading } = useAuth();
  const router = useRouter();

  useEffect(() => {
    if (!loading && !user) router.push('/login');
  }, [user, loading]);

  if (loading || !user) return null;
  return <div>Protected content</div>;
}
```

Accessing auth state

```
import { useAuth } from '../contexts/AuthContext';

const { user, token, login, logout, loading } = useAuth();
```

user is the decoded JWT payload ({ id, name, email, role }), or null if not logged in.

Running Tests

Tests live in server/src/__tests__/.

```
# From repo root or server/:
cd server
npm test          # run once
npm run test:watch # watch mode
```

The test runner is Jest with ts-jest. NODE_ENV=test is set automatically by the npm script.

Git Workflow

main stable history, base for all feature branches
release Vercel production deployment target

Never commit directly to release. The workflow is:

1. Branch from main: git checkout -b feature/my-thing main
2. Build and test locally
3. Open a PR to main
4. Once merged and verified, merge main release to trigger a Vercel deployment

The release branch auto-deploys to Vercel on every push. Only merge into it when the feature is fully tested.

Troubleshooting Common Issues

"Cannot connect to MongoDB"

- Check MONGODB_URI in .env — must be MONGODB_URI, not MONGO_URI
- Ensure your Atlas cluster's IP allowlist includes your current IP (or 0.0.0.0/0 for dev)
- Test the connection string in MongoDB Compass before pasting it into .env

API calls return 404 in development

- The backend must be running on port 5001 — check that npm run dev started both processes
- If only the frontend started, run cd server && npm run dev in a second terminal

JWT errors / "Not authorized"

- JWT_SECRET must match between .env and whatever the server is using — restart the server

after changing it
- Tokens expire — log out and back in if you get a 401 after JWT_EXPIRE has passed

Email not sending in development

- This is expected if email vars are not set. Look for a line like Preview URL: https://ethereal.email/message/... in the server terminal output
- If you need real email delivery in dev, sign up for Mailtrap (free) and add its SMTP credentials to .env

TypeScript errors after pulling

```
cd server && npm run build   # check for compile errors
cd client && npx tsc --noEmit
```

Pulling changes sometimes requires reinstalling dependencies:

```
npm run install-all
```
